■ Task Management Application Source Code

1. ■■ package.json

■■ ■■■■■ ■■■■■■■■■■ ■■ ■■■■ ■■■■■■■■ ■■ ■■■■■■■■■■■■ ■■ ■■■■■■ ■■■■■ ■■■■

```
{
  "name": "task-management-app",
  "version": "1.0.0",
  "description": "Full-stack Task Management Application",
  "main": "server.js",
  "scripts": {
    "start": "node server.js"
  },
  "dependencies": {
    "bcryptjs": "^2.4.3",
    "express": "^4.19.2",
    "jsonwebtoken": "^9.0.2"
  }
}
```

2. ■■ server.js

■■ ■■■■■ ■■■■■■■ (Backend API) ■■■■■■ ■■, ■■ ■■■■■■ ■■■■■■■■■■■■ ■■ ■■■■■■ ■■■■■■■■■ (CRUD) ■■■■■■■■ ■■■■

```
const express = require('express');
const path = require('path');
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');

const app = express();
const PORT = process.env.PORT || 3000;
const JWT_SECRET = 'your_super_secret_jwt_key_123';

app.use(express.json());
app.use(express.static(path.join(__dirname, 'public')));

let users = [];
let tasks = [];

const authenticateToken = (req, res, next) => {
    const authHeader = req.headers['authorization'];
    const token = authHeader && authHeader.split(' ')[1];
    if (!token) return res.status(401).json({ message: 'Access Denied' });

    jwt.verify(token, JWT_SECRET, (err, user) => {
        if (err) return res.status(403).json({ message: 'Invalid Token' });
        req.user = user;
        next();
    });
};

app.post('/api/auth/signup', async (req, res) => {
    try {
        const { username, email, password } = req.body;
        if (users.find(u => u.email === email)) {
```

```
            return res.status(400).json({ message: 'User already exists' });
        }
        const hashedPassword = await bcrypt.hash(password, 10);
        const newUser = { id: Date.now().toString(), username, email, password: hashedPassword };
        users.push(newUser);
        res.status(201).json({ message: 'User registered' });
    } catch { res.status(500).json({ message: 'Error registering user' }); }
});

app.post('/api/auth/login', async (req, res) => {
    const { email, password } = req.body;
    const user = users.find(u => u.email === email);
    if (!user) return res.status(400).json({ message: 'User not found' });
    try {
        if (await bcrypt.compare(password, user.password)) {
            const token = jwt.sign({ id: user.id, username: user.username }, JWT_SECRET, { expiresIn: '1
            res.json({ token, username: user.username });
        } else { res.status(400).json({ message: 'Invalid credentials' }); }
    } catch { res.status(500).json({ message: 'Error logging in' }); }
});

app.get('/api/tasks', authenticateToken, (req, res) => {
    res.json(tasks.filter(t => t.userId === req.user.id));
});

app.post('/api/tasks', authenticateToken, (req, res) => {
    const { title, description } = req.body;
    if (!title) return res.status(400).json({ message: 'Title is required' });
    const newTask = { id: Date.now().toString(), userId: req.user.id, title, description: description ||
    tasks.push(newTask);
    res.status(201).json(newTask);
});

app.put('/api/tasks/:id', authenticateToken, (req, res) => {
    const task = tasks.find(t => t.id === req.params.id && t.userId === req.user.id);
    if (!task) return res.status(404).json({ message: 'Task not found' });
    const { title, description, completed } = req.body;
    if (title !== undefined) task.title = title;
    if (description !== undefined) task.description = description;
    if (completed !== undefined) task.completed = completed;
    res.json(task);
});

app.delete('/api/tasks/:id', authenticateToken, (req, res) => {
    const idx = tasks.findIndex(t => t.id === req.params.id && t.userId === req.user.id);
    if (idx === -1) return res.status(404).json({ message: 'Task not found' });
    tasks.splice(idx, 1);
    res.json({ message: 'Task deleted successfully' });
});

app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
```

3. ■ public/index.html

■■ ■■■■ ■■■■■■■■■ ■■■■ ■■■■■■■■■ (UI) ■■, ■■■■■ ■■■■■■■ ■■ ■■■■ ■■■■■■ ■■ ■■■■ ■■■■■■■■■■■■ ■■■■■■ ■■■■ ■■■■

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
    <title>Task Manager</title>
    <script src="https://cdn.jsdelivr.net/npm/@tailwindcss/browser@4"></script>
</head>
<body class="bg-gray-100 min-h-screen flex items-center justify-center p-4">
    <div id="auth-section" class="bg-white p-8 rounded-xl shadow-md w-full max-w-md">
        <h2 id="auth-title" class="text-2xl font-bold text-center mb-6">Login</h2>
        <form id="auth-form" class="space-y-4">
            <div id="username-container" class="hidden">
                <label class="block text-sm font-semibold">Username</label>
                <input type="text" id="auth-username" class="w-full border p-2 rounded">
            </div>
            <div>
                <label class="block text-sm font-semibold">Email</label>
                <input type="email" id="auth-email" required class="w-full border p-2 rounded">
            </div>
            <div>
                <label class="block text-sm font-semibold">Password</label>
                <input type="password" id="auth-password" required class="w-full border p-2 rounded">
            </div>
            <button type="submit" id="auth-btn" class="w-full bg-blue-600 text-white p-2 rounded font-bo
        </form>
        <p class="text-sm text-center mt-4"><button id="auth-toggle-btn" class="text-blue-600 underline"
    </div>

    <div id="app-section" class="hidden w-full max-w-2xl bg-white rounded-xl shadow-md p-6">
        <div class="flex justify-between items-center border-b pb-4 mb-6">
            <h1 class="text-2xl font-bold">■ Welcome, <span id="user-display" class="text-blue-600">User
            <button id="logout-btn" class="bg-red-500 text-white px-3 py-1 rounded">Logout</button>
        </div>
        <form id="task-form" class="bg-gray-50 p-4 rounded-lg mb-6 border space-y-3">
            <input type="text" id="task-title" placeholder="Task Title *" required class="w-full border
            <textarea id="task-desc" placeholder="Task Description" class="w-full border p-2 rounded"></
            <button type="submit" class="w-full bg-green-600 text-white py-2 rounded font-bold">Add Task
        </form>
        <div id="tasks-list" class="space-y-3"></div>
    </div>
    <script src="app.js"></script>
</body>
</html>
```

4. ■ public/app.js

■■ ■■■■■ ■■■■ ■■■■■■■■ (UI) ■■ ■■■■■■ ■■ ■■■■■■■■■ ■■■■■■ ■■ ■■■■■■■ ■■ API ■■ ■■■■■ ■■■■■ ■■ ■■■■ ■■■■■ ■■■■

```
const API_URL = '/api';
let token = localStorage.getItem('token');
let isLoginMode = true;

const authSection = document.getElementById('auth-section');
const appSection = document.getElementById('app-section');
const authForm = document.getElementById('auth-form');
const authTitle = document.getElementById('auth-title');
const authBtn = document.getElementById('auth-btn');
const usernameContainer = document.getElementById('username-container');
const authToggleBtn = document.getElementById('auth-toggle-btn');
const userDisplay = document.getElementById('user-display');
const logoutBtn = document.getElementById('logout-btn');
const taskForm = document.getElementById('task-form');
const tasksList = document.getElementById('tasks-list');

authToggleBtn.addEventListener('click', () => {
```

```
    isLoginMode = !isLoginMode;
    authTitle.innerText = isLoginMode ? 'Login' : 'Sign Up';
    authBtn.innerText = isLoginMode ? 'Login' : 'Sign Up';
    usernameContainer.classList.toggle('hidden', isLoginMode);
});

authForm.addEventListener('submit', async (e) => {
    e.preventDefault();
    const email = document.getElementById('auth-email').value;
    const password = document.getElementById('auth-password').value;
    const username = document.getElementById('auth-username').value;
    const endpoint = isLoginMode ? '/auth/login' : '/auth/signup';
    const payload = isLoginMode ? { email, password } : { username, email, password };

    try {
        const res = await fetch(`${API_URL}${endpoint}`, {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify(payload)
        });
        const data = await res.json();
        if (!res.ok) throw new Error(data.message);
        if (isLoginMode) {
            localStorage.setItem('token', data.token);
            localStorage.setItem('username', data.username);
            token = data.token;
            checkAuth();
        } else {
            alert('Signup Successful!');
            authToggleBtn.click();
        }
    } catch (err) { alert(err.message); }
});

logoutBtn.addEventListener('click', () => {
    localStorage.clear(); token = null; checkAuth();
});

function checkAuth() {
    if (token) {
        authSection.classList.add('hidden');
        appSection.classList.remove('hidden');
        userDisplay.innerText = localStorage.getItem('username');
        loadTasks();
    } else {
        authSection.classList.remove('hidden');
        appSection.classList.add('hidden');
    }
}

async function loadTasks() {
    const res = await fetch(`${API_URL}/tasks`, { headers: { 'Authorization': `Bearer ${token}` } });
    renderTasks(await res.json());
}

function renderTasks(tasks) {
    tasksList.innerHTML = tasks.length === 0 ? '<p class="text-gray-400 text-center">No tasks found.</p>
    tasks.forEach(task => {
        const div = document.createElement('div');
        div.className = `flex justify-between items-center p-4 border rounded bg-white`;
        div.innerHTML = `
            <div>
                <h4 class="font-bold ${task.completed ? 'line-through' : ''}">${task.title}</h4>
```

```
                <p class="text-sm text-gray-500">${task.description}</p>
            </div>
            <div class="flex gap-2">
                <button onclick="toggleComplete('${task.id}', ${task.completed})" class="bg-gray-200 px-
                <button onclick="deleteTask('${task.id}')" class="bg-red-100 text-red-600 px-2 py-1 text
            </div>`;
        tasksList.appendChild(div);
    });
}

checkAuth();
```